

# Relatório AOOP

## AOOP A1 - AR Face Filters

### Autores

Paulo Simões, Nº 31377

Francisco Matos, Nº 31406

# 1. Introdução

O projeto **AR Face Filters** consiste numa aplicação web que permite aplicar filtros faciais em tempo real através da webcam. A ideia surgiu com base nas aplicações de realidade aumentada que são comuns em redes sociais, como filtros de rosto, efeitos visuais e elementos gráficos que acompanham os movimentos da face.

O objetivo principal foi criar uma aplicação interativa que conseguisse detetar a face do utilizador, identificar pontos importantes do rosto e usar essa informação para posicionar filtros de forma dinâmica. Ao longo do desenvolvimento foram exploradas várias componentes, desde a deteção facial com MediaPipe até à renderização de filtros 2D, efeitos em canvas e modelos 3D com Three.js.

A aplicação final permite selecionar filtros por categoria, combinar filtros compatíveis, ajustar a sua posição, escala e rotação, guardar presets e tirar fotografias com os filtros aplicados.

## 2. Objetivos do projeto

O principal objetivo deste trabalho foi desenvolver uma aplicação de filtros faciais em tempo real, funcional diretamente no navegador.

De forma mais concreta, os objetivos definidos foram:

- Utilizar a webcam para captar vídeo em direto.
- Detetar a face do utilizador em tempo real.
- Identificar landmarks faciais para saber onde colocar os filtros.
- Aplicar filtros 2D em zonas como cabeça, olhos, boca e face.
- Integrar filtros 3D através de modelos GLB.
- Criar efeitos visuais desenhados diretamente em canvas.
- Permitir ajustes manuais dos filtros.
- Guardar combinações de filtros através de presets.
- Capturar uma fotografia com os filtros aplicados.

Estes objetivos foram sendo desenvolvidos de forma progressiva, começando pela deteção facial e filtros mais simples, e avançando depois para funcionalidades mais complexas, como modelos 3D, efeitos visuais e captura final da imagem.

### 3. Tecnologias utilizadas

A aplicação foi desenvolvida com **HTML5**, **CSS3** e **JavaScript**, sem recurso a frameworks externas de interface.

Para a deteção facial foi utilizado o **MediaPipe Tasks Vision**, através do modelo **FaceLandmarker**. Esta ferramenta permite detetar a face do utilizador e obter vários pontos de referência, que depois são usados para calcular posições, distâncias e rotações.

Para os filtros 2D e efeitos visuais foi utilizado **Canvas 2D**. Esta tecnologia foi usada para desenhar imagens, formas, partículas e efeitos diretamente sobre a imagem captada pela webcam.

Para os filtros 3D foi utilizado **Three.js**, juntamente com **WebGL**. Esta parte foi necessária para carregar e renderizar modelos 3D em formato **GLB**, como os óculos 3D e o chapéu 3D.

Também foi utilizado **LocalStorage** para guardar ajustes dos filtros e presets criados pelo utilizador. Desta forma, algumas configurações mantêm-se mesmo depois de atualizar a página.

### 4. Estrutura do projeto

O projeto segue a estrutura indicada para o repositório, com as pastas principais `src`, `data`, `docs`, `notebooks` e o ficheiro `README.md`.

A pasta mais importante é a pasta `src`, onde se encontra a aplicação. Dentro desta pasta estão os ficheiros principais:

- `index.html`, responsável pela estrutura da interface.
- `style.css`, responsável pelo aspeto visual da aplicação.
- `app.js`, responsável pela lógica principal da aplicação.

Dentro de `src/js`, o código foi dividido por responsabilidades. A pasta `config` contém a configuração dos filtros e categorias. A pasta `core` contém funções auxiliares para trabalhar com landmarks e posições da face. A pasta `render` contém os módulos responsáveis por desenhar os filtros 2D, filtros 3D e efeitos visuais.

A pasta `models` contém o ficheiro `face_landmarker.task`, usado pelo MediaPipe para a deteção facial. A pasta `assets` contém os recursos da aplicação, como imagens de filtros, miniaturas e modelos 3D.

As pastas `data` e `notebooks` ficaram reservadas para eventuais dados ou experiências futuras. Neste projeto, não foi necessário utilizá-las, uma vez que a

aplicação trabalha diretamente com a webcam e usa um modelo pré-treinado do MediaPipe.

## 5. Funcionamento geral da aplicação

Quando a aplicação é iniciada, começa por carregar o modelo de deteção facial e pedir acesso à webcam. Depois de a webcam estar ativa, cada frame do vídeo é analisado pelo MediaPipe.

Sempre que uma face é detetada, o MediaPipe devolve landmarks faciais. Estes landmarks vêm em coordenadas normalizadas, por isso são convertidos para coordenadas do canvas. A partir daí, a aplicação consegue calcular pontos importantes, como o centro dos olhos, a distância entre os olhos, a posição da boca e os limites da face.

Com estes dados, os filtros selecionados são posicionados no local correto. Os filtros 2D são desenhados no canvas principal, os filtros 3D são renderizados num canvas próprio com WebGL e os efeitos visuais também são desenhados em canvas.

A aplicação atualiza tudo em tempo real, acompanhando os movimentos do utilizador enquanto a webcam está ativa.

## 6. Filtros 2D

Os filtros 2D foram a primeira parte visual implementada. Estes filtros são aplicados diretamente sobre o canvas e acompanham a posição da face.

Alguns filtros usam imagens, como chapéu, coroa, chapéu de Natal, chapéu de bruxa, máscara, bigode e óculos. Outros filtros foram desenhados diretamente por código, como o **Pixel Glasses** e a **Neon Mask**.

Para posicionar os filtros, foram usados landmarks específicos. Por exemplo, os filtros de olhos usam a posição dos dois olhos, os filtros de boca usam os pontos dos lábios e os filtros de cabeça usam o contorno da face.

A rotação dos filtros é calculada através da inclinação entre os dois olhos. Assim, quando o utilizador inclina a cabeça, o filtro acompanha esse movimento.

## 7. Filtros 3D

Uma das partes mais desafiantes do projeto foi a integração de filtros 3D. Foram implementados dois filtros principais: **Óculos 3D** e **Chapéu 3D**.

Estes filtros usam modelos em formato GLB, carregados e renderizados com Three.js. Ao contrário dos filtros 2D, os modelos 3D precisam de ser posicionados num espaço tridimensional, o que tornou esta parte mais complexa.

Para definir o tamanho dos modelos, foi usada principalmente a distância entre os olhos. Esta distância permite adaptar a escala do modelo à face do utilizador. Para a posição, foram usadas zonas como os olhos e o topo da cabeça.

Também foram adicionados controlos específicos para os filtros 3D, incluindo rotação nos eixos X e Y. Isto foi útil porque os modelos GLB podem vir com orientações diferentes, sendo necessário ajustar manualmente a sua posição e rotação.

Foi ainda implementada uma suavização do movimento. Esta melhoria reduziu pequenos tremores nos filtros 3D, tornando o acompanhamento da face mais estável.

## 8. Efeitos visuais

Para além dos filtros tradicionais, foram implementados efeitos visuais desenhados diretamente em canvas.

Foram criados três efeitos principais:

- **Glitch**, que cria distorções e deslocamentos sobre a zona da face.
- **Inferno**, que gera partículas de fogo na parte superior da face.
- **Matrix**, que desenha caracteres sobre o rosto, criando um efeito visual inspirado no filme Matrix.

Estes efeitos não usam imagens externas. São gerados diretamente por código, o que permitiu explorar melhor as possibilidades do canvas em tempo real.

A existência destes efeitos deu mais variedade à aplicação e permitiu criar combinações mais interessantes com filtros 2D e 3D.

## 9. Compatibilidade entre filtros

Durante o desenvolvimento, foi necessário controlar melhor a forma como os filtros podiam ser combinados. Se todos os filtros pudessem ser ativados ao mesmo tempo, surgiam situações pouco coerentes, como vários chapéus ou vários óculos sobrepostos.

Para resolver isso, foi criada uma lógica de compatibilidade por zonas. Cada filtro pertence a uma zona, como cabeça, olhos, boca, face completa ou efeitos.

Quando o utilizador seleciona um novo filtro, a aplicação verifica se já existe outro filtro ativo na mesma zona. Se existir, o anterior é removido. Desta forma, é possível combinar filtros de zonas diferentes, como **Chapéu 3D + Óculos 3D + Matrix**, mas evita-se ter dois filtros da mesma zona ao mesmo tempo.

Esta lógica tornou a utilização da aplicação mais organizada e melhorou bastante o resultado visual.

## 10. Ajustes e presets

A aplicação permite ajustar cada filtro individualmente. Os ajustes disponíveis incluem tamanho, posição horizontal, posição vertical e rotação. Nos filtros 3D existem ainda ajustes adicionais para a rotação nos eixos X e Y.

Estes ajustes são guardados no LocalStorage, para que se mantenham mesmo depois de atualizar a página.

Também foi criado um sistema de presets. Um preset guarda a combinação de filtros selecionados e os ajustes de cada um. Assim, o utilizador pode criar uma combinação, guardá-la e voltar a aplicá-la rapidamente.

Esta funcionalidade é útil tanto para o utilizador como para a demonstração da aplicação, porque permite recuperar rapidamente estilos já preparados.

## 11. Captura de fotografia

A aplicação inclui uma funcionalidade para tirar fotografias com os filtros aplicados.

Para isso, é criado um canvas temporário onde são combinadas várias camadas. Primeiro é desenhada a imagem da webcam. Depois é desenhado o canvas dos filtros 3D. Por fim, é desenhado o canvas principal, onde estão os filtros 2D e os efeitos visuais.

Esta abordagem foi necessária porque os filtros 3D são renderizados num canvas separado. Se fosse capturada apenas a webcam, os modelos 3D não apareceriam na fotografia final.

Depois da captura, a imagem pode ser vista em pré-visualização, aberta em maior dimensão e transferida em formato PNG.

## 12. Limitações

Apesar de a aplicação estar funcional, existem algumas limitações.

A aplicação foi pensada para detetar apenas uma face de cada vez. Em situações com várias pessoas na câmara, apenas uma face é considerada.

Outra limitação está relacionada com os filtros 3D. Em movimentos normais da cabeça, os filtros acompanham bem a face. No entanto, em rotações mais fortes, o alinhamento pode deixar de ser perfeito, porque o posicionamento é feito com base nos landmarks e não com uma reconstrução 3D completa da face.

Também não foi implementada oclusão facial real. Isto significa que os modelos 3D aparecem sempre por cima da imagem da webcam, mesmo em situações em que parte do objeto deveria ficar atrás da face.

A qualidade da webcam e a iluminação também influenciam o resultado, porque afetam a precisão da deteção facial.

## 13. Melhorias futuras

Como trabalho futuro, uma das principais melhorias seria utilizar diretamente a matriz de transformação facial fornecida pelo MediaPipe. Isso poderia melhorar o alinhamento dos modelos 3D e tornar a rotação mais precisa.

Outra melhoria importante seria implementar oclusão facial. Com uma face mesh invisível, seria possível tornar os modelos 3D mais realistas, fazendo com que algumas partes ficassem escondidas atrás da face quando necessário.

Também poderiam ser adicionados mais modelos 3D, suporte a múltiplas faces, exportação de presets e uma melhor adaptação da interface para dispositivos móveis.

## 14. Conclusão

Com este projeto foi possível desenvolver uma aplicação web funcional de realidade aumentada, capaz de aplicar filtros faciais em tempo real através da webcam.

A aplicação combina várias tecnologias: MediaPipe para detecção facial, Canvas 2D para filtros e efeitos visuais, Three.js e WebGL para modelos 3D, e LocalStorage para guardar ajustes e presets.

Durante o desenvolvimento foram explorados vários desafios, principalmente relacionados com o posicionamento dos filtros, a rotação da cabeça, a integração dos modelos 3D e a captura da imagem final com várias camadas.

Apesar de existirem limitações, o resultado final cumpre os objetivos definidos e demonstra uma aplicação prática de visão por computador, renderização gráfica e interação em tempo real no navegador.